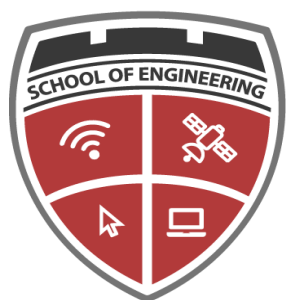




UNIVERSIDAD AUTÓNOMA “GABRIEL RENE MORENO”
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y
TELECOMUNICACIONES

“UAGRM SCHOOL OF ENGINEERING”



UAGRM
SCHOOL OF
ENGINEERING
UNIDAD DE POSTGRADO

MAESTRÍA EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

“Proyecto final”

MODULO:

ML-Ops y Puesta en Producción

ESTUDIANTE:

Ing. Alex Junior Paco Surco

Santa Cruz, Bolivia
Junio, 2026



1. Problema y objetivo

El abandono de clientes (churn) afecta ingresos y costos de retención. El proyecto entrena un modelo offline y lo expone mediante una API en Docker. El objetivo operativo es que equipos externos consuman predicciones sin acceder al código ni al archivo del modelo, y que sea posible auditar qué versión del modelo está activa en cada momento.

2. Mejora técnica incorporada — GET /info

La mejora elegida es un endpoint adicional GET /info que expone metadatos del modelo desplegado: versión, autor, variables de entrada, algoritmo, fecha de entrenamiento, archivo del modelo y métricas offline de referencia. Responde la pregunta MLOps: ¿qué artefacto está sirviendo esta API en producción?

¿Por qué no basta con /health?

- /health confirma que el servicio está vivo y que modelo_cargado es true.
- /health no indica la versión del modelo, ni las variables esperadas, ni cuándo fue entrenado.
- Ante un incidente o degradación de predicciones, /info reduce el tiempo de diagnóstico.

Campos expuestos por /info:

- version_modelo y version_servicio — identificación del artefacto y del servicio API.
- variables y variable_objetivo — contrato de datos de entrada y salida esperada.
- entrenado_en — referencia temporal para evaluar posible desactualización o drift.
- algoritmo y archivo_modelo — trazabilidad técnica del artefacto cargado.
- metricas (F1, recall, precision, accuracy) — expediente offline del entrenamiento, no métricas en tiempo real.

Implementación:

- Función info_modelo() en api/main.py lee modelo_churn_v1_metadata.json al iniciar.
- Disponible en ejecución local (puerto 8000) y en Docker (puerto 8080).
- Incluido en la documentación Swagger (/docs) como mejora personal verificable.

3. API y endpoints — evidencia de /info

Comparación /health vs /info:

GET /health (solo disponibilidad):

```
{
  "status": "ok",
  "modelo_cargado": true,
  "archivo_modelo": "modelo_churn.pkl",
  "autor": "Alex J. Paco Surco",
  "variables": [
    "antigüedad",
    "cargo_mensual",
    "reclamos"
  ]
}
```



```
]
}
```

GET /info (mejora — trazabilidad completa):

```
{
  "autor": "Alex J. Paco Surco",
  "version_servicio": "1.0.0",
  "version_modelo": "v1",
  "algoritmo": "RandomForestClassifier",
  "variables": [
    "antiguedad",
    "cargo_mensual",
    "reclamos"
  ],
  "variable_objetivo": "churn",
  "entrenado_en": "2026-06-14T22:53:03Z",
  "archivo_modelo": "modelo_churn.pkl",
  "metricas": {
    "accuracy": 0.5666666666666667,
    "precision": 0.575,
    "recall": 0.71875,
    "f1_score": 0.6388888888888888
  }
}
```

Pretty-print ☐

```
{"status": "ok", "modelo_cargado": true, "archivo_modelo": "modelo_churn.pkl", "autor": "Alex J. Paco Surco", "variables": ["antiguedad", "cargo_mensual", "reclamos"]}
```

Figura 1. /health — confirma que el modelo está cargado, sin identidad del artefacto



Pretty-print ☐

```
{
  "autor": "Alex J. Paco Surco",
  "version_servicio": "1.0.0",
  "version_modelo": "v1",
  "algoritmo": "RandomForestClassifier",
  "variables": [
    "antiguedad",
    "cargo_mensual",
    "reclamos"
  ],
  "variable_objetivo": "churn",
  "entrenado_en": "2026-06-14T22:53:03Z",
  "archivo_modelo": "modelo_churn.pkl",
  "metricas": {
    "accuracy": 0.5666666666666667,
    "precision": 0.575,
    "recall": 0.71875,
    "f1_score": 0.6388888888888888
  }
}
```

Figura 2. /info — mejora técnica con versión, variables, fecha y métricas offline

API Predictiva de Churn 1.0.0 OAS 3.1

/openapi.json

Interfaz controlada para consumir el modelo de abandono de clientes sin acceso directo al código ni al archivo del modelo.

default ^

GET	/	Read Root	▼
GET	/health	Health Check	▼
GET	/info	Info Modelo	▼
POST	/predict	Predict	▼

Schemas ^

- ClienteChurnRequest > Expand all object
- HTTPValidationError > Expand all object
- PredictResponse > Expand all object
- ValidationError > Expand all object

Figura 3. Swagger /docs — /info visible como endpoint adicional

4. Ejecución dentro de Docker

Imagen: churn-api-surco | Contenedor: churn-container-surco | Puerto: 8080→8000



La mejora /info debe estar disponible dentro del contenedor. URL de verificación:

http://127.0.0.1:8080/info

Imagen construida:

```
IMAGE          ID          DISK USAGE  CONTENT SIZE  EXTRA
churn-api-surco:latest  af4f638b5f2f    705MB      160MB        U
WARNING: This output is designed for human readability. For machine-readable output,
please use --format.
```

Contenedor activo:

```
CONTAINER ID  IMAGE          COMMAND                  CREATED        STATUS
PORTS
b2fdb68c87bb  churn-api-surco  "uvicorn api.main:apâ€¦"  2 hours ago   Up 2 hours
(healthy)    0.0.0.0:8080->8000/tcp, [::]:8080->8000/tcp  churn-container-surco
```

Registros del contenedor:

```
INFO:      172.17.0.1:43426 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:43438 - "GET /info HTTP/1.1" 200 OK
INFO:      172.17.0.1:43450 - "POST /predict HTTP/1.1" 200 OK
INFO:      172.17.0.1:43464 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      172.17.0.1:43480 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      172.17.0.1:43482 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      172.17.0.1:43492 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      127.0.0.1:45722 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:58088 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:44432 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:51764 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:57046 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:44332 - "GET / HTTP/1.1" 200 OK
INFO:      172.17.0.1:44348 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:44350 - "GET /info HTTP/1.1" 200 OK
INFO:      172.17.0.1:44366 - "POST /predict HTTP/1.1" 200 OK
INFO:      172.17.0.1:44374 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      172.17.0.1:44382 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      172.17.0.1:44390 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
INFO:      172.17.0.1:44404 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
```

5. Propuesta de monitoreo aplicada

Alerta prioritaria: /health caído o /info devuelve versión inesperada.

Acción: revisar contenedor, logs y coherencia entre metadata y modelo desplegado.

Métrica seleccionada	Tipo	Señal de alerta	Acción propuesta
Estado de /health	Técnica	El endpoint no responde o modelo_cargado=false	Revisar contenedor Docker y logs; validar carga del modelo
Disponibilidad de /info	Técnica	/info no responde o devuelve versión distinta a la esperada	Verificar despliegue correcto y archivo modelo_churn_v1_metadata.json



Latencia de /predict	Técnica	Tiempo de respuesta p95 > 500 ms de forma sostenida	Revisar recursos del contenedor
F1-score offline (referencia en /info)	Modelo	Caída sostenida tras reentrenamiento comparado con baseline en /info	Evaluar calidad del nuevo artefacto antes de promover versión

6. Error o incidente

Síntoma	El contenedor churn-container-surco se detuvo al iniciar; /health y /info no respondían.
Posible causa	Incompatibilidad entre el modelo serializado (numpy 2.x / scikit-learn 1.8) y versiones antiguas en la imagen Docker.
Forma de detección	docker ps sin contenedor activo; docker logs con ModuleNotFoundError numpy._core.numeric.
Evidencia que revisaría	docker logs, requirements.txt, GET /health y GET /info (una vez restaurado el servicio).
Acción correctiva	Alinear requirements.txt, reconstruir imagen churn-api-surco y validar /info devuelve version_modelo esperada.
Acción preventiva	Tras cada build, verificar /health y /info antes de dar por cerrado el despliegue.

7. Drift y respuesta operativa

Riesgo identificado	Incremento sostenido del cargo_mensual promedio respecto al dataset de entrenamiento.
Tipo de drift	Data drift
Impacto	Las predicciones pueden degradarse; /info permite saber qué versión del modelo estaba activa al detectar la señal.
Señal de alerta	Cambio en variables de entrada y métricas de modelo; comparar entrenado_en en /info con antigüedad del drift.
Respuesta operativa	Reentrenar, actualizar metadata, desplegar nueva versión y confirmar en /info que version_modelo cambió a v2.

8. Conclusión

La mejora GET /info eleva la API desde un servicio que solo predice hacia un componente MLOps auditable: permite identificar el modelo activo, sus variables y su fecha de entrenamiento sin acceder al repositorio. Complementa /health (disponibilidad) con



trazabilidad del artefacto, facilitando monitoreo, diagnóstico de incidentes y gestión de nuevas versiones tras reentrenamiento.

9. Enlace al repositorio GitHub

<https://github.com/alepaco-maton/churn-api-mlops>